

Лабораторная работа

**Механизмы межпроцессового
взаимодействия и синхронизации
процессов в QNX Neutrino**

Методические указания

1 Цель работы

Изучение основных механизмов межпроцессного взаимодействия и синхронизации в ОС *QNX Neutrino*.

2 Механизмы межпроцессного взаимодействия

Процессы операционной системы *QNX Neutrino*, в каких бы "родственных" отношениях они ни находились, выполняются каждый в своем изолированном адресном пространстве и не могут обмениваться информацией без использования механизмов *межпроцессного взаимодействия* (*IPC*, *InterProcess Communication*). В *QNX Neutrino* реализован ряд таких механизмов, как стандартных, так и уникальных:

- очереди сообщений *POSIX* (реализованы в администраторе очередей *mqueue*);

- именованные программные каналы (реализованы в администраторе файловой системы *POSIX*);

- неименованные программные каналы (реализованы в администраторе каналов *pipe*);

- разделяемая память;

Кроме того, мы поговорим об именованных и неименованных семафорах.

2.1 Очереди сообщений POSIX

Очереди сообщений *POSIX* реализованы в *QNX* с помощью администратора очередей *mqueue*.

Для строгого соответствия с *POSIX*, очередь сообщений нужно создавать с именем, начинающегося с обратного слеша (/) и не содержать других слешей. Но следует помнить, что расширение *POSIX* стандарта поддерживает пути которые содержат несколько слешей. Это позволяет, например, компании размещать все свои очереди сообщений под своим именем, и быть уверенными, что имена ее очереди не будут конфликтовать с другими компаниями.

В ОС *QNX Neutrino*, все очереди сообщений размещаются в пространстве файловых имен в следующих директориях:

- */dev/mqueue* при использовании традиционной (*mqueue*) реализации
- */dev/mq* при использовании альтернативной (*mq*) реализации.

Пример, с традиционной реализацией (*mqueue*):

имя, задаваемое в функции <i>mq_open()</i>	Путь очереди сообщений
<i>/data</i>	<i>/dev/mqueue/data</i>
<i>/acme/data</i>	<i>/dev/mqueue/acme/data</i>
<i>/qnx/data</i>	<i>/dev/mqueue/qnx/data</i>

Просмотреть все очереди сообщений в системе можно с помощью команды *ls* следующим образом:

```
ls -Rl /dev/mqueue
```

Размер файлового объекта указывает количество сообщений в очереди.

Сообщения *POSIX* не копируются непосредственно из адресного пространства одного процесса в адресное пространство другого, как это делают сообщения *QNX*, а используют промежуточное адресное пространство администратора *mqueue*. Поэтому сообщения *POSIX* работают относительно медленно.

Очереди сообщений *POSIX* управляются посредством следующих функций:

Функция	Описание
---------	----------

<code>mq_open()</code>	Открытие очереди сообщений
<code>mq_close()</code>	Закрытие очереди сообщений
<code>mq_unlink()</code>	Удалить очередь сообщений
<code>mq_send()</code>	Добавить сообщение в очередь сообщений
<code>mq_receive()</code>	Получить сообщение из очереди сообщений
<code>mq_notify()</code>	Сообщить вызывающему процессу, что в очереди сообщений доступно сообщение
<code>mq_setattr()</code>	Установить атрибуты очереди сообщений
<code>mq_getattr()</code>	Получить атрибуты очереди сообщений

2.2 Программные каналы

Каналы, или *программные каналы*, — это один из традиционных способов *IPC*. Для использования каналов в *QNX Neutrino* должен быть запущен *pipe* - администратор программных каналов. Назначение канала — обеспечить однонаправленную передачу данных от одного процесса к другому. При этом вывод одного процесса соединяется с вводом другого.

Различия между именованными и неименованными каналами:

именованные каналы представляют собой особый тип файла, хранящегося в файловой системе (т. е. один процесс пишет данные в этот файл, а другой — читает), поэтому именованные каналы работают медленнее, но могут использоваться для *IPC* между любыми процессами в сети;

неименованные каналы реализованы с помощью администратора каналов *pipe*, который и выполняет буферизацию данных;

неименованные каналы могут использоваться для *IPC* только между процессами, связанными отношениями "родительский" — "дочерний".

Несмотря на то, что неименованный канал работает быстрее именованного, он уступает по скорости *QNX*-сообщениям.

2.2.1 Неименованные программные каналы

Неименованный программный канал - это механизм, который командный интерпретатор использует для создания конвейеров. Этот механизм реализован в администраторе ресурсов *pipe* и обеспечивает передачу данных через промежуточный буфер в оперативной памяти (в адресном пространстве администратора *pipe*). Размер буфера определен константой *PIPE_BUF* в заголовочном файле *<limits.h>*. Неименованный канал (*pipe*) уничтожается после закрытия его сторон. Функция *pathconf()* возвращает значение размера буфера.

Неименованный канал создается функцией *pipe()*, которой в качестве аргумента передается массив из двух целых чисел. В этот массив записывается два файловых дескриптора, один из которых в последующем используется для записи информации, другой - для чтения.

Типичный способ использования канала - соединение выхода одного процесса с входом другого процесса. Это соединение часто выполняется в командной строке. Например:

```
ls | more
```

направит стандартный вывод команды *ls* на стандартный вход утилиты *more*, посредством канала.

Чтобы:

Создать канал из командного интерпретатора

Используется:

Символ канала ("|")

Создать канал программным путем

функции *pipe()* или *popen()*

К достоинствам неименованных программных каналов можно отнести то,

что они являются стандартным **POSIX**-механизмом **IPC**, а также то, что это - достаточно быстрый механизм. К недостаткам - то, что обмен данными возможен только между процессами-"родственниками", т. к. процесс может получить файловые дескрипторы для работы с неименованным каналом, только наследуя их от родительского процесса.

2.2.2 Именованные программные каналы

Именованные каналы - это **POSIX**-механизм, поддержка которого реализована в файловой системе **Neutrino**. В основе механизма лежит особый файл – типа **FIFO**, выполняющего функцию буфера для данных.

Чтобы:

Используется:

Создать именованный канал (**FIFO**) из Команда **mkfifo**
командного интерпретатора

Создать именованный канал (**FIFO**) функцией **mkfifo()**
программным путем

Удалить именованный канал (**FIFO**) из Утилита **rm**
командного интерпретатора

Удалить именованный канал программным функцией **remove()** или
путем **unlink()**

Поскольку **FIFO** - это файл, имеющий имя, то, во-первых, работа с ним выполняется практически так же, как с обычным файлом (**open()**, **read()**, **write()**, **close()** и т. п.), во-вторых, именованный канал медленнее неименованного, но данные, записанные в именованный канал, сохраняются в случае сбоя (например, отключения питания).

2.3 Разделяемая память

Разделяемая память (**shared memory**) обеспечивает наивысшую пропускную способность среди механизмов межпроцессной коммуникации (**IPC**). После создания объекта разделяемой памяти, процесс, имеющий доступ к этому объекту может, используя указатель на этот объект, напрямую читать и записывать данные туда. Это означает, что доступ к разделяемой памяти осуществляется несинхронно. Если процесс обновляет области разделяемой памяти, необходимо соблюдать осторожность, чтобы предотвратить чтение или обновление некоторой области другим процессом. Даже в простейшем случае чтения, другой процесс может получить информацию неверную или испорченную информацию.

Для решения этих проблем, разделяемая память часто используется в сочетании с одним из примитивов синхронизации, чтобы произвести обновление данных процессами атомарно. Однако, если размер обновлений мал, тогда примитивы синхронизации будут ограничивать изначально высокую пропускную способность разделяемой памяти. Разделяемая память наиболее эффективна, когда используется обновление больших размеров данных.

В качестве примитивов синхронизации для разделяемой памяти подходят как семафоры, так и мьютексы. Семафоры внедрены в стандарт реального времени **POSIX** для межпроцессной синхронизации, в то время как мьютексы были внедрены в стандарт **POSIX** как стандарт синхронизации потоков. Мьютексы могут также использоваться между потоками в различных процессах. **POSIX** считает это дополнительной возможностью, **QNX Neutrino** это полностью обеспечивает. Мьютексы более эффективны, чем семафоры.

2.3.1 Создание объектов разделяемой памяти

Множество потоков внутри процесса разделяют память этого пространства. Для совместного использования памяти между процессами, сначала необходимо создать область разделяемой памяти (**shared-memory region**) и затем отобразить (**map**) этот регион в адресное пространство процессов. Области разделяемой

памяти создаются и управляются посредством следующих функций:

Функция	Описание
<code>shm_open()</code>	Открыть (или создать) область разделяемой памяти
<code>shm_close()</code>	Закрыть область разделяемой памяти
<code>mmap()</code>	Отобразить область разделяемой памяти в адресное пространство процесса
<code>munmap()</code>	Отменить отображение (<i>unmap</i>) области разделяемой памяти на адресное пространство процесса
<code>mprotect()</code>	Изменяет атрибуты защищенности области разделяемой памяти
<code>msync()</code>	Синхронизирует содержимое памяти с физическим носителем
<code>shm_unlink()</code>	Удаление области разделяемой памяти
<code>shm_ctl()</code>	Задаются нужные атрибуты разделяемого региона

Разделяемая память *POSIX* реализована в ОС *QNX Neutrino* посредством администратора процессов (*procnto*). Вызовы, приведенные выше, реализованы как сообщения к администратору процессов (*procnto*).

Функция `shm_open()` принимает те же аргументы, что и функция `open()` и возвращает файловый дескриптор на объект. Как и обычный файл, эта функция позволяет создать новый объект разделяемой памяти, или открыть существующий объект разделяемой памяти.

Файловый дескриптор необходимо открыть для чтения. Если необходимо записать данные в объект памяти, также необходимо указать флаг (*MAP_PRIVATE*).

Когда новый объект разделяемой памяти создается, размер объекта устанавливается равным нулю. Чтобы задать размер, вы можете использовать функцию `truncate()` - ту же самую функцию, что и для установки размера файла, или функцию `shm_ctl()`.

Если объект разделяемой памяти имеет файловый дескриптор, можно использовать функцию `mmap()` для отображения объекта, или части этого объекта в адресное пространство процесса. Функция `mmap()` является краеугольным камнем управления памятью в ОС *QNX Neutrino* и заслуживает подробного обсуждения ее возможностей.

Функция `mmap()` определена следующим образом:

```
void * mmap( void *where_i_want_it,
             size_t length,
             int memory_protections,
             int mapping_flags,
             int fd,
             off_t offset_within_shared_memory );
```

Функция производит отображение в адресное пространство процесса `length` байт разделяемой памяти по смещению `offset_within_shared_memory` из объекта разделяемой памяти, связанной с файловым дескриптором `fd`.

Функция `mmap()` будет отображать содержимое памяти по адресу `where_i_want_it` в адресном пространстве процесса. Памяти будет предоставлена защита определяемая параметрами `memory_protections` и отображение будет осуществляться в соответствии с учетом флага `mapping_flags`.

Три аргумента: `fd`, `offset_within_shared_memory`, и `length` определяют часть конкретного объекта разделяемой памяти, которая будет отображаться в адресное пространство. Часто отображается весь объект целиком; в этом случае смещение `offset_within_shared_memory` будет

нулевым, а длина **length** будет равна размеру объекта разделяемой памяти в байтах. На процессорах **Intel**, длина будет определяться числом, кратным размеру страницы, который составляет 4096 байт.

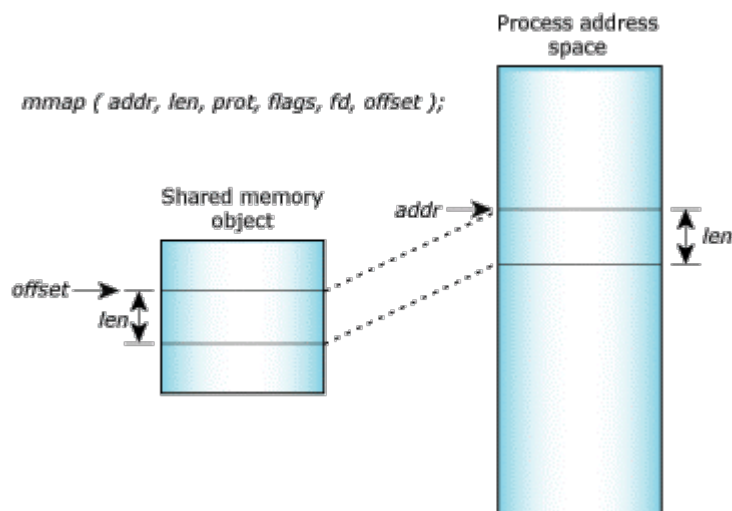


Рис. 1. Отображение (mapping) память с помощью функции **mmap()**

Возвращаемое функцией **mmap()** значение является адресом, в адресном пространстве процесса, куда отображен объект. Аргумент **where_i_want_it** используется системой, в качестве указания на то, куда именно объект стоит разместить. Если это возможно, объект будет размещен по требуемому адресу. Большинство приложений указывает адрес равным нулю, что дает системе возможность самостоятельно определить адрес для размещения объекта.

В качестве атрибута защиты **memory_protections** используется битовая маска, которая может содержать несколько флагов:

Декларация	Описание
PROT_EXEC	Содержимое памяти может быть исполнено
PROT_NOCACHE	Содержимое памяти не должно быть кэшировано
PROT_NONE	Доступ запрещен
PROT_READ	Доступ на чтение
PROT_WRITE	Доступ на запись

Флаги **mapping_flags** определяют как память будет отображаться. Эти флаги разделяются на две части - первая часть означает тип флага и должна быть одним из следующих действий:

Тип отображения	Описание
MAP_SHARED	Отображение может быть разделяемым на несколько процессов; изменения передаются обратно базовому объекту
MAP_PRIVATE	Отображение используется только вызывающим объектом; изменения не распространяются к базовому объекту. Функция mmap() выделяет системную память и создает копию объекта

Тип **MAP_SHARED** используется для создания разделяемой памяти между несколькими процессами; **MAP_PRIVATE** имеет более специализированное применение.

Некоторые флаги могут быть установлены (при помощи логической операции ИЛИ) в указанный ранее тип для более точного определения типа отображения. Они подробно описаны в описании функции **mmap()**. Приведем также несколько интересных флагов:

MAP_ANON

Анонимное отображение памяти, которая не связана ни с одним файловым дескриптором; параметр **fd** необходимо указать как **NOFD**. Функция **mmap()** выделяет память и, по умолчанию, заполняет выделенную память нулями.

Обычно используют `MAP_ANON` с флагом `MAP_PRIVATE`, но также его можно использовать с флагом `MAP_SHARED` для создания области разделяемой памяти для клонируемых приложений. Также `MAP_ANON` используется как база для страничного распределения памяти.

`MAP_FIXED`

Отображение объекта по адресу, на который указывает `where_i_want_it`. Если область разделяемой памяти содержит указатели, тогда эту область, возможно, потребуется расположить по одному и тому же адресу в адресных пространствах всех процессов, отображающих ее. Этого можно избежать, используя смещение внутри области разделяемой памяти вместо прямых указателей.

`MAP_PHYS`

Этот флаг показывает, что происходят операции с физической памятью. Параметр `fd` должен быть установлен в `NOFD`. Когда используется без `MAP_ANON`, смещение `offset_within_shared_memory` указывает точный физический адрес отображения. Если используется с `MAP_ANON`, то должна быть выделена область непрерывной физической памяти.

Также можно использовать флаги `MAP_NOX64K` и `MAP_BELOW16M` для дальнейшего определения метода выделения памяти типа `MAP_ANON` и ограничений адресов, которые существуют в некоторых формах `DMA`.

`MAP_NOX64K`

Используется с `MAP_PHYS` | `MAP_ANON`. Выделенная область памяти не будет пересекать границы в 64 Кб. Это необходимо для старых 16-битных контроллеров `DMA`.

`MAP_BELOW16M`

Используется вместе с `MAP_PHYS` | `MAP_ANON`. Выделенная область памяти не будет занимать в физической памяти более 16 Мб. Это необходимо при использовании `DMA` контроллера с устройствами на `ISA` шине.

`MAP_NOINIT`

Пример 1.

Рассмотрим пример из двух программ - `shm_creator` и `shm_user`. Пример работает так:

Запускается программа `shm_creator`, которая создает регион разделяемой памяти, задает его параметры и отображает на него некий буфер, содержащий текстовую строку.

Запускается программа `shm_user`, которая отображает регион разделяемой памяти, созданный программой `shm_creator`, на свой буфер и печатает содержимое этого буфера.

Приведем исходные тексты обеих программ. Текст файла `shm_creator.c` выглядит так:

```

#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
int main()
{
    int fd, status;
    void* buffer;
    fd=shm_open("/swd_es", O_RDWR | O_CREAT, 0777);
    if (fd==-1)
    {
        perror("shm_creator");
        return EXIT_FAILURE;
    }
    status=ftruncate(fd, 100);
    if (status!=0)
    {
        perror("shm_creator");
        return EXIT_FAILURE;
    }
    Buffer=mmap(0, 100, PROT_READ | PROT_WRITE, MAP_SHARED,
fd, 0);
    if (buffer==MAP_FAILED)
    {
        perror("shm_creator");
        return EXIT_FAILURE;
    }
    sprintf(buffer, "It's a nice day today, isn't it?");
    printf("shm_creator: %s\n", buffer);
    return EXIT_SUCCESS;
}

```

Сначала вызываем функцию `shm_open()`:

```
fd=shm_open("/swd_es", O_RDWR|O_CREAT, 0777);
```

Первый аргумент `"/swd_es"` - имя региона разделяемой памяти (или, как еще говорят, разделяемого объекта памяти);

Примечание

Когда имя разделяемого объекта начинается с символа `/`, объект будет помещен в служебный каталог `/dev/shmem`. То есть реальное имя создаваемого нами региона - `/dev/shmem/swd_es`.

Второй аргумент представляет собой битовую маску из нескольких флагов, к этим флагам относятся:

`O_RDONLY` - открыть объект только для чтения;

`O_RDWR` - открыть объект для чтения и записи;

`O_CREAT` - создать разделяемый объект с режимом доступа, заданным третьим аргументом функции `shm_open()`. Если объект уже существует, то флаг `O_CREAT` игнорируется, за исключением случаев, когда указан еще и флаг `O_EXCL`;

`O_EXCL` - этот флаг используют совместно с флагом `O_CREAT`. В результате, если разделяемый объект памяти уже существует, то функция `shm_open()` завершится с ошибкой;

`O_TRUNC` - этот флаг работает, когда объект уже существует и успешно открыт для чтения/записи. При этом размер объекта становится равным нулю (режим доступа и идентификатор владельца сохраняются).

Третий аргумент задает атрибуты доступа к разделяемому объекту.

Функция вернет файловый дескриптор `fd`, который в последующем и будет

использоваться для доступа к данному разделяемому объекту.

Теперь нужно сделать, чтобы разделяемый объект имел нужный размер и параметры. Для этого используют функцию `shm_ctl()`. Иногда вместо функции `shm_ctl()` используют функцию `ftruncate()`.

```
ftruncate(fd, 100);
```

В качестве первого аргумента используется тот самый идентификатор объекта разделяемой памяти `fd`, который вернула функция `shm_open()`.

Теперь созданный объект, имеющий нужный размер, необходимо отобразить на виртуальное адресное пространство нашего процесса:

```
buffer=mmap(0, 100, PROT_READ|PROT_WRITE, MAP_SHARED, fd, 0);
```

Первый и последний аргументы в этом примере не нужны - они требуются при работе с физической памятью. Второй аргумент (`100`) указывает, какой величины фрагмент разделяемого объекта стоит отобразить на адресное пространство процесса (отобразить весь объект). Третий аргумент представляет собой атрибут защиты `memory_protections` (см. выше):

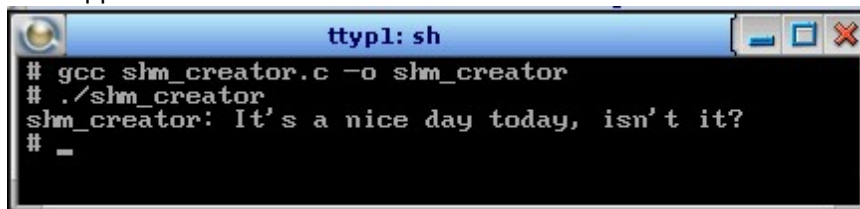
Четвертый аргумент флаг `mapping_flags` определяет режим отображения региона. В данном случае лучше задать `MAP_SHARED` (остальные режимы используются для работы с физической памятью).

Пятый аргумент - идентификатор разделяемого объекта.

Функция `mmap()` возвращает указатель на область виртуальной памяти процесса, на который отображен разделяемый объект (`buffer`). Все, что будет записано по этому адресу, будет отображено на разделяемый объект (конечно же, столько байт, сколько было задано в функции `mmap()`). Запишем в буфер текстовую строку:

```
sprintf(buffer, "It's a nice day today, isn't it?");
```

Вывод:



```
tty1: sh
# gcc shm_creator.c -o shm_creator
# ./shm_creator
shm_creator: It's a nice day today, isn't it?
# _
```

Теперь посмотрим на исходный текст программы `shm_user.c`:

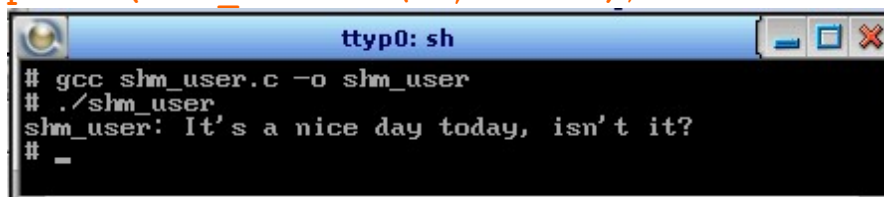
```

#include <stdlib.h>
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
int main()
{
    int fd;
    char *buffer;
    fd=shm_open("/swd_es", O_RDONLY, 0777);
    if (fd== -1)
    {
        perror("shm_user");
        return EXIT_FAILURE;
    }
    buffer=mmap(0, 100, PROT_READ, MAP_SHARED, fd, 0);
    if (buffer==MAP_FAILED)
    {
        perror("shm_user");
        return EXIT_FAILURE;
    }
    printf("shm_user: %s\n", buffer);
    munmap(buffer, sizeof (buffer));
    return EXIT_SUCCESS;
}

```

Как видно из текста программы, для получения доступа к разделяемому объекту снова используется функция `shm_open()`. Для того чтобы отобразить разделяемый регион на адресное пространство процесса, используется функция `mmap()`. В результате получаем указатель `buffer` на область виртуальной памяти процесса, который можно использовать. Распечатаем содержимое разделяемого объекта:

```
printf("shm_user: %s\n", buffer);
```



По аналогии можно передавать между процессами любые структуры данных. За правильность интерпретации данных, содержащихся в разделяемой памяти, отвечает программист.

Как процесс, считывающий данные из разделяемой памяти, определяет, что запись данных уже закончена и данные готовы для чтения? Ответ: никак. Чтобы избежать нарушений целостности данных, нам нужно использовать механизмы *синхронизации*.

2.4 Механизмы синхронизации POSIX

QNX Neutrino предоставляет широкий набор **POSIX**-элементов синхронизации на уровне потоков. Некоторые из этих примитивов синхронизации могут использоваться для взаимодействия между потоками в разных процессах. Некоторые сервисы синхронизации перечислены ниже:

Сервисы синхронизации:

Semaphores (семафоры)

Mutexes (мьютексы)

Condvars (условные переменные)
Barriers (барьеры)
Sleepon locks (ждущие блокировки)
Reader/writer locks (блокировки чтения/записи)
FIFO scheduling (*FIFO*-планировка)
Send/Receive/Reply (Отправка/прием/ответ)
Atomic operations (Атомарные операции)

2.4.1 Семафоры

Принято рассматривать семафоры двух видов: бинарные, счетчик которых может принимать только значения 0 либо 1, и счетные, или простые, счетчик которых может принимать большие положительные значения. В *QNX* 6.x максимальное значение счетчика определяется переменной *SEM_VALUE_MAX*, значение которой равно 32 767.

Принципиально схема работы семафора позволяет использовать его для решения максимально широкого круга задач. Приведем только несколько схематичных примеров:

□ **Ожидание условия (уведомление).** Часто возникает необходимость остановить (заблокировать) поток до тех пор, пока не наступит некое событие (выполнится условие). Разблокировать остановленный поток в таком случае может только другой, активный к этому времени поток. Предварительно инициализируем семафор нулевым значением:

Поток А

```
while (!expression)
    sem_wait(&sem);
```

Поток В

```
expression = true;
sem_post(&sem);
```

В этом случае поток *А* ожидает выполнения некоего условия (операция *sem_wait()*), а поток *В* уведомляет (операция *sem_post()*) о выполнении условия любые ожидающие этого условия потоки.

□ **Взаимное исключение.** С помощью семафора можно решить и другую классическую проблему синхронизации - безопасное совместное исполнение кода. Эта проблема возникает, когда из нескольких потоков необходимо провести модификацию общих переменных или обратиться к одному системному ресурсу. В таком случае необходимо установить четкий порядок обращений, не допускающий одновременности исполнения соответствующего участка кода (взаимное исключение). Здесь семафор инициализируется единицей:

Поток А

```
sem_wait(&sem);
/* код, который нужно
защитить от совместного
использования */
sem_post(&sem);
```

Поток В

```
sem_wait(&sem),
/* код, который нужно
защитить от совместного
использования */
sem_post(&sem);
```

В данном случае последовательность доступа потоков к защищаемому участку кода (такой участок кода еще называют **критической секцией**) не играет никакой роли; здесь важно не допустить, чтобы два потока исполняли этот код одновременно.

Поскольку семафор инициализируется единицей, первый вызов *sem_wait()* не приводит к блокированию потока, а лишь понижает счетчик семафора до нуля. В таком положении семафора любой поток, повторно вызвавший *sem_wait()* на входе в критическую секцию, будет заблокирован до тех пор, пока первый вошедший поток не покинет этот участок кода и не вызовет *sem_post()*. После этого **один** из потоков, ожидающих увеличения счетчика семафора, получит управление (это будет один поток, и нельзя заранее сказать, какой именно) и выполнит операцию декремента над счетчиком, вновь заблокировав вход в критическую секцию. Таким образом можно гарантировать

строго последовательное выполнение такого кода.

□ **Классическая задача последовательного доступа по типу производитель/потребитель.** Такая ситуация возникает, когда один поток передает другому данные через общую переменную. Пока производитель не «положит» новые данные в эту переменную, потребитель должен простаивать в ожидании.

Приведем классическое решение этой задачи. В этом случае нам понадобится два семафора (по одному на каждый поток), которые должны инициализироваться следующим образом: тот, который защищает чтение (потребление), инициализируется нулем (блокирует читающий поток), а тот, который защищает запись (производство), - единицей (открывает доступ «писателю» к общему ресурсу).

Поток А

```
sem_wait(&sem_A);  
/* критическая секция */  
sem_post(&sem_B);
```

Поток В

```
sem_wait(&sem_B),  
/* критическая секция */  
sem_post(&sem_A);
```

Положим, поток **А** является производителем данных, необходимых для работы потока **В**. Соответственно семафор **sem_A** инициализирован **1**, а семафор **sem_B** инициализирован **0**. Когда поток **В** попытается обратиться к общей переменной за данными для работы, он будет блокирован в ожидании результатов работы потока **А**. Поток **А**, подготовив необходимые данные, войдет в критическую секцию (поскольку его семафор разблокирован), установит новые данные и, покидая критическую секцию, разблокирует семафор потока **В**. После этого поток **В** будет разблокирован и сможет получить новые данные. Если данные производятся в цикле (а это обычная ситуация), то поток **А** не сможет повторно получить доступ к общей переменной до тех пор, пока поток **В** не закончит чтение этой переменной и не покинет критическую секцию.

□ **Счетный семафор, позволяющий контролировать количество обращений,** допускает его применение в весьма специфических задачах, где другие средства синхронизации требуют от программиста дополнительной работы.

Проиллюстрируем работу счетного семафора. Предположим, нам надо синхронизировать работу двух потоков так, чтобы один из них всегда шел «вслед» за другим, то есть выполнял некие циклические операции только после их выполнения первым потоком. Для реализации этой схемы нам понадобится семафор, инициализированный нулевым значением.

Поток А

```
while(true)  
{  
/* работа А; */  
sem_post(&sem);  
}
```

Поток В

```
while(true)  
{  
/* sem_wait(&sem); */  
работа В;  
}
```

Поток **А** в каждом цикле увеличивает счетчик семафора на 1, а поток **В** в свою очередь стремится «выбрать слабину» и в каждом цикле уменьшает этот счетчик на единицу до тех пор, пока он не достигнет нуля. Вне зависимости от приоритетов, дисциплины диспетчеризации или любых источников блокировки потоков поток **В** будет «следовать» за потоком **А** и выполнять не больше циклов, чем его «ведущий» поток.

Эти примеры показывают главную особенность семафора: любой поток может менять его состояние (внутренний счетчик), тем самым делая это элемент синхронизации идеальным средством управления порядком выполнения потоков. Семафор является идеальным инструментом для построения собственных средств планирования (диспетчеризации) выполнения потоков вашей системы. Стандарт **POSIX** предусматривает наличие специальных «именованных»

семафоров, к которым может обращаться любой поток, принадлежащий любому процессу, выполняющемуся в системе, а в ОС **QNX** к семафорам может обращаться и любой поток любого процесса во всей сети **QNX**. Таким образом, семафор позволяет осуществлять планирование выполнения задач даже для распределенной сетевой системы.

Функции работы с семафорами объявлены в заголовочном файле **<semaphore.h>**.

Именованные семафоры

Именованные семафоры, при прочих равных условиях, медленнее и требуют для своей работы запущенного в системе менеджера очередей сообщений **POSIX (mqqueue)**.

Создание именованного семафора:

```
sem_t * mySem;  
mySem=sem_open("/my_semaphore", O_CREAT, 0777, 5);
```

Функция **sem_open()** принимает в качестве аргумента имя семафора (в нашем случае - **/dev/sem/my_semaphore**), флаг создания семафора, атрибуты доступа к семафору и значение счетчика. Значение счетчика задает количество потоков, которые могут владеть семафором, не блокируясь.

Примечание

Если речь об однопоточных приложениях, то фраза "количество потоков" равнозначна фразе "количество процессов".

Идея заключается в том, что если мы хотим ограничить количество потоков, работающих с ресурсом, числом **5**, то мы создаем такой семафор **mySem**.

Закрытие именованного семафора:

```
sem_close(mySem)
```

В вопросе подключения и отключения работа с именованным семафором аналогична работе с обычным файлом. Для именованных семафоров существует операция **sem_unlink()**, аналогичная операции **unlink()** для обычного файла.

Неименованные семафоры

Создание неименованного семафора:

```
sem_t * mySem;  
sem_init(mySem, 1, 5);
```

В этом примере мы получим семафор, на который ссылается указатель **mySem** со счетчиком **5** (как и в предыдущем примере). Обратите, пожалуйста, внимание на второй целочисленный аргумент **1**. На самом деле **sem_init()** различает только два значения этого аргумента - "ноль" и "не ноль". Если задать **0**, то семафор может быть использован только для синхронизации потоков *внутри* одного процесса. Если задать не **0**, то семафор можно будет разделять с другими процессами данной ЭВМ посредством механизма разделяемой памяти.

Разрушение неименованного семафора:

```
sem_destroy(mySem)
```

Использование неименованного семафора полностью идентично использованию именованного семафора.

Операции над семафорами

Для семафора определены три модификации операции блокировки:

```
int sem_wait(sem_t* sem);  
int sem_trywait(sem_t* sem);  
#include <time.h>  
int sem_timedwait(sem_t* sem, const struct timespec *  
abs_timeout);
```

Функция простого ожидания **sem_wait()** пытается выполнить декремент счетчика семафора. Если исходное значение счетчика было больше или равно 1, то функция после выполнения операции декремента возвращает управление в вызвавший код. Если же значение внутреннего счетчика семафора равнялось 0,

выполнение вызвавшего функцию потока блокируется до тех пор, пока какой-либо другой поток не увеличит значение счетчика семафора. После того как значение счетчика будет увеличено, заблокированный поток получает управление (в порядке очереди), выполняет (завершает) декремент счетчика семафора и возвращает управление. Блокированное состояние потока может быть также прервано получением направленного ему сигнала.

Функция ожидания с проверкой семафора, `sem_trywait()`, до выполнения операции над семафором проверяет значение счетчика. Если это значение больше нуля, функция выполняет декремент счетчика, а если значение равно нулю - возвращает управление потоку, не блокируясь на ожидании доступности семафора (но захват семафора в этом случае не происходит, о чем извещает код возврата).

Функция ожидания с тайм-аутом, `sem_timedwait()`, ожидает возможности уменьшить на 1 счетчик семафора (блокируется на ожидании) до момента времени `abs_timeout`.

Операция освобождения:

```
int sem_post(sem_t* sem);
```

Эта операция увеличивает на единицу (инкремент) внутренний счетчик семафора, и в этом она диаметрально противоположна операции блокировки. Если перед этим значение счетчика семафора было равно 0, то один из потоков, ожидающих разблокирования семафора, переходит в состояние готовности. Из списка ожидающих потоков система выбирает самый приоритетный, а если таких несколько, то поток, дольше всех ждавший из очереди наиболее приоритетных потоков.

Получение статуса семафора:

```
int sem_getvalue(sem_t* sem, int* value);
```

Эта функция используется преимущественно для отладки операций над семафорами. По адресу, указанному в `value`, устанавливается текущее значение счетчика семафора. Поскольку значение счетчика семафора может измениться в любой момент, то значение, которое возвращает эта функция, имеет смысл только непосредственно в точке ее вызова.

Семафор является крайне гибким и эффективным средством синхронизации, особенно удобным для построения собственных средств планирования выполнения потоков. В этом смысле семафор представляет ценность не только как самостоятельное средство синхронизации выполнения потоков, но и как материал для построения специфических средств планирования и синхронизации для конкретных задач.

2.4.2 Мьютексы

Мьютекс (от *MUTual EXclusion* - взаимное исключение) - это один из базовых примитивов синхронизации *QNX Neutrino*. Мьютекс используется для обеспечения монопольного доступа к разделяемым между потоками данным. Этот элемент реализуется на уровне микроядра системы и имеет широкий набор атрибутов и настроек.

Параметры мьютекса хранятся в структуре `pthread_mutexattr_t`, которая определена типом `sync_attr_t`. Эта структура должна быть, создана и определена до инициализации мьютекса, после чего может быть переопределена и использована для других объектов типа мьютекс.

Инициализация мьютекса:

```
int pthread_mutexattr_init(const pthread_mutexattr_t* attr);
```

Функция инициализирует структуру атрибутов мьютекса, на которую указывает параметр `attr`. Тип данных `pthread_mutexattr_t` определен в файле `<pthread.h>` (производный от типа `sync_attr_t`, который в свою очередь определен в файле `<target_nto.h>`) следующим образом:


```

struct _sync_attr_t
{
    int protocol;
    int flags;
    int pnoceiling;
    int clockid; /* только для condvar */
    int reserved[4];
};

```

После инициализации значения по умолчанию могут быть прочитаны или изменены группой функций, приведенной ниже.

```

int pthread_mutex_init(pthread_mutex_t* mutex, const
pthread_mutexattr_t* attr);

```

Структура данных `pthread_mutex_t` определена в файле `<pthread.h>` (производный тип от типа `sync_t`, который в свою очередь определен в файле `<target_nto.h>`) и имеет следующий вид:

```

struct _sync_t
{
    /* Счетчик для рекурсивного мьютекса или семафора
    nt count;
    /* TID потока - имеет смысл и применяется только для
    мьютексов */
    unsigned owner;
}

```

Функция `pthread_mutex_init()` инициализирует переданный объект мьютекс в соответствии со значением переданных атрибутов. Если вместо `attr` передать `NULL`, то мьютекс будет создан в соответствии со значениями атрибутов по умолчанию.

Вместо прямого вызова функции `pthread_mutex_init()` для начальной инициализации статических мьютексов (глобальных на уровне файла кода или пространства имен namespace либо явно описанных с квалификатором `static`) можно воспользоваться двумя макросами `PTHREAD_MUTEX_INITIALIZER` и `PTHREAD_RMUTEX_INITIALIZER`:

```

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t mutex = PTHREAO_RMUTEX_INITIALIZER;

```

Первый из них создает мьютекс в соответствии со значениями атрибутов по умолчанию, а второй - мьютекс с разрешенным рекурсивным захватом.

Определение протокола защиты от инверсии приоритетов:

```

int pthread_mutexattr_setprotocol(pthread_mutexattr_t* attr,
int protocol);
int pthread_mutexattr_getprotocol(pthread_mutexattr_t* attr,
int* protocol);

```

Эти функции устанавливают/считывают протокол, который реализуется мьютексом для защиты от инверсии приоритетов. Переменная `protocol` может принимать следующие значения:

`PTHREAD_PRIO_INHERIT` (значение по умолчанию) - определяет, что для воспрепятствования возникновению инверсии приоритетов будет использоваться протокол наследования приоритетов.

`PTHREAD_PRIO_PROTECT` - любой поток, захвативший мьютекс и созданный с таким параметром, будет устанавливать фиксированный уровень приоритета в соответствии со значением поля `priocelimg`, возвращаемого функцией `pthread_mutexattr_getprioceiling()`. Таким образом, установка этого значения в качестве протокола мьютекса приводит к реализации протокола граничного приоритета для защиты от инверсии приоритетов.

Определение типа мьютекса:

```
int pthread_mutexattr_settype(pthread_mutexattr_t attr, int
type);
int pthread_mutexattr_gettype(const pthread_mutexattr_t*
attr, int* type);
```

В версиях QNX 6.2.1 и 6.3 предусматривается создание мьютексов следующих типов:

- **PTHREAD_MUTEX_NORMAL** - для этого типа не проводится контроль “мертвой блокировки” (*deadlock*) в ситуации, когда поток, захвативший мьютекс, пытается захватить его повторно. Поэтому при попытке повторного захвата такого мьютекса тем же потоком этот поток будет безусловно блокирован (то есть он попадает в “мертвую блокировку”, а это во всех случаях аварийная ситуация в выполнении приложения); такой мьютекс уже некому разблокировать (мьютекс может разблокироваться только своим владельцем). Попытка освободить (*unlock*) мьютекс такого типа, захваченный другим потоком, или освободить незахваченный мьютекс ни к чему не приводит, при этом не возвращается ошибка выполнения.

- **PTHREAD_MUTEX_ERRORCHECK** - включается контроль ошибок. В этом режиме регистрируются следующие ситуации:

- попытка повторного захвата мьютекса тем же потоком;
- попытка освобождения мьютекса, захваченного другим потоком;
- освобождение свободного мьютекса.

- **PTHREAD_MUTEX_RECURSIVE** - мьютекс, допускающий рекурсивный захват. Поток, пытающийся захватить мьютекс, уже захваченный в этом потоке, сможет это сделать, при этом количество захватов будет учитываться при освобождении мьютекса. Другой поток сможет захватить такой мьютекс только тогда, когда он будет освобожден столько же раз, сколько был захвачен. Если поток пытается освободить мьютекс, захваченный другим потоком, или свободный мьютекс, то будет возвращено сообщение об ошибке (регистрируются ошибки, предусмотренные предыдущим типом, за исключением повторного захвата, который является для рекурсивного мьютекса штатным действием).

- **PTHREAD_MUTEX_DEFAULT** (значение по умолчанию) - попытка рекурсивного захвата мьютекса, освобождения мьютекса, захваченного другим потоком, или освобождения уже свободного мьютекса ни к чему не приводит и не возвращает ошибку выполнения.

Разрушение объекта мьютекс:

```
int pthread_mutexattr_destroy(pthread_mutexattr_t* attr);
```

Вызов разрушает ранее применявшийся объект - атрибутивную запись мьютекса, после чего она уже не может более использоваться для инициализации мьютекса без предварительного выполнения вызова **pthread_mutexattr_init()**.

На этом обсуждение атрибутов заканчивается, и мы переходим непосредственно к функциям работы с мьютексом.

```
int pthread_mutex_destroy(pthread_mutex_t* mutex);
```

Вызов разрушает объект мьютекс, на который указывает переменная **mutex**. После чего эта переменная не может быть использована без предварительного вызова **pthread_mutex_init()**.

Захват мьютекса может производиться тремя разными функциями.

Простой захват:

```
int pthread_mutex_lock(pthread_mutex_t* mutex);
```

Функция захватывает мьютекс, на который ссылается **mutex**. Если мьютекс уже захвачен другим потоком, то вызвавший поток блокируется до освобождения мьютекса и после этого захватывает его. Только после этого функция **pthread_mutex_lock()** возвращает управление. Если захватить мьютекс пытается поток, который им уже владеет, то поведение функции

`pthread_mutex_lock()` будет зависеть от значений атрибутов мьютекса, указанных при его создании. При создании мьютекса с параметрами по умолчанию попытка повторного захвата мьютекса ни к чему не приводит. Если включен режим контроля ошибок и отключен рекурсивный захват мьютекса, функция `pthread_mutex_lock()` возвращает `EDEADLK` при попытке повторного захвата мьютекса тем же потоком.

Попытка захвата:

```
int pthread_mutex_trylock(pthread_mutex_t* mutex),
```

Функция проверяет, свободен ли мьютекс `mutex`, и если да, то она захватывает его. В противном случае функция возвращает значение `EBUSY`.

Захват с установкой времени ожидания:

```
int pthread_mutex_timedlock(pthread_mutex_t* mutex, const struct timespec* abs_timeout);
```

Функция проверяет, свободен ли мьютекс `mutex`, и если да, то поток, в котором вызвана функция, захватывает этот мьютекс. Если мьютекс уже захвачен, вызвавший поток блокируется до освобождения мьютекса либо до наступления времени, указанного в аргументе `abs_timeout`. Если это время уже наступило, поток не блокируется вообще, но захват все-таки произойдет, если мьютекс свободен.

Наступление времени определяется по часам `REALTIME_CLOCK`, когда значение часов оказывается равным или большим значения, указанного в `abs_timeout`. Тип данных `timespec` определен в файле `<time.h>`.

Если мьютекс создан с атрибутом протокола `PRIO_INHERIT`, то после выхода потока из блокировки на мьютексе по тайм-ауту приоритет владельца мьютекса подвергается пересмотру в соответствии с приоритетами потоков, оставшихся в очереди на захват мьютекса.

Освобождение мьютекса:

```
int pthread_mutex_unlock(pthread_mutex_t* mutex);
```

Функция `pthread_mutex_unlock()` освобождает мьютекс, на который ссылается переменная `mutex`. Вызвавший поток должен быть владельцем мьютекса. Если есть потоки, заблокированные в ожидании освобождения мьютекса, то поток с наивысшим приоритетом (или при равных приоритетах дольше всех ждавший) выходит из заблокированного состояния и становится владельцем мьютекса.

Для мьютексов, разрешающих рекурсивный захват, функция освобождения должна вызываться столько же раз, сколько и функция захвата.

2.4.3 Условные переменные

Условные переменные (*CONDition VARiable*) используется для блокировки потока в критической секции пока удовлетворяется некоторое условие. Условие может быть сколь угодно сложным и не зависеть от условной переменной. Однако, условная переменная должна всегда использоваться с мьютексом для проверки условия.

Инициализация параметров:

```
int pthread_condattr_init(pthread_condattr_t* attr);
```

Функция инициализирует структуру атрибутов условной переменной, на которую указывает параметр `attr`. Структура данных `pthread_condattr_t` определена в файле `<pthread.h>` и является производной от типа `syncattr_t`. При инициализации атрибуты устанавливаются в значения по умолчанию.

Для условной переменной возможна модификация значительно меньшего числа параметров, чем для мьютекса. Следующие функции описывают доступ к этим параметрам.

Параметр доступа:

```

int pthread_condattr_setpshared(pthread_condattr_t* attr,
int pshared);
int pthread_condattr_getpshared(const pthread_condattr_t*
attr, int* pshared);

```

Функции устанавливают/считывают, возможен ли доступ к условной переменной из потоков, порожденных в других процессах. Параметр **pshared** может принимать следующие значения:

- **PTHREAD_PROCESS_SHARED** - любой поток, имеющий доступ к области памяти, в которой расположена условная переменная, может ее использовать.
- **PTHREAD_PROCESS_PRIVATE** (значение по умолчанию) - только поток, созданный в контексте того же процесса, в котором находится условная переменная, может ее использовать. Если поток из другого процесса попытается использовать условную переменную, созданную с параметром **PTHREAD_PROCESS_PRIVATE**, результат будет неопределен.

Параметры тайм-аута:

```

int pthread_condattr_setclock(pthread_condattr_t* attr,
clockid_t id);
int pthread_condattr_getclock(const pthread_condattr_t*
attr, clockid_t* id);

```

Функции устанавливают/считывают, каким способом (т. е. на основании какого счетчика) вычисляется значение тайм-аута при вызовах **pthread_cond_timedwait()**. Этот параметр в [QNX Neutrino 6.2](#) не является обязательным и введен в соответствии со стандартом [POSIX 1003.1j \(draft\)](#). На практике можно устанавливать только значение **REALTIME_CLOCK** в качестве параметра **id**; это же значение является значением по умолчанию.

Разрушение блока параметров:

```

int pthread_condattr_destroy(pthread_condattr_t* attr);

```

Функция разрушает блок параметров условной переменной, на которые указывает **attr**, после чего он уже не может использоваться без повторной инициализации.

На практике разрушение параметров объекта синхронизации не имеет особого смысла. Вы всегда можете переопределить атрибуты, содержащиеся в переменной **attr**, для инициализации других условных переменных.

Инициализация условной переменной:

```

pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
int pthread_cond_init(pthread_cond_t* cond,
pthread_condattr_t* attr),

```

Инициализирует условную переменную **cond** со значениями, установленными атрибутами **attr**. Вместо прямого вызова функции **pthread_cond_init()** для начальной инициализации статических условных переменных (глобальных на уровне файла кода или пространства имен namespace либо явно описанных с квалификатором **static**) можно воспользоваться макросом **PTHREAD_COND_INITIALIZER**.

Простое ожидание условия:

```

int pthread_cond_wait(pthread_cond_t* cond, pthread_mutex_t*
mutex);

```

Вызов функции блокирует вызвавший поток на условной переменной **cond** и разблокирует мьютекс **mutex**. Поток блокируется до тех пор, пока другой поток не вызовет функцию разблокирования на условной переменной **cond** (**pthread_cond_signal()** или **pthread_cond_broadcast()**). Мьютекс **mutex** должен быть захвачен потоком до вызова функции. Поток, заблокированный на условной переменной, может быть разблокирован также приходом сигнала или вызовом завершения потока. В любом случае при разблокировании потока и

выходе из функции ожидания поток вновь захватывает мьютекс `mutex`.

Не следует использовать условную переменную с мьютексом, у которого разрешен рекурсивный захват.

Ожидание условия с тайм-аутом:

```
int pthread_cond_timedwait(pthread_cond_t* cond,
pthread_mutex_t* mutex, const struct timespec* abstime);
```

Поведение функции идентично варианту обычного ожидания, за исключением того, что ожидание может завершиться также при наступлении времени, переданного параметром `abstime`.

После наступления времени тайм-аута управление совсем не обязательно вернется к вызвавшему потоку. После наступления этого времени функция переведет поток из состояния блокирования на условной переменной в состояние готовности и предпримет попытку захвата мьютекса. Если мьютекс в это время захвачен другим потоком, вызвавший поток перейдет в состояние блокирования на мьютексе.

Выполнение условия:

Штатным способом разблокирования потока, блокированного на условной переменной, является вызов функции, сигнализирующей о выполнении условия: `pthread_cond_signal()` или `pthread_cond_broadcast()`. Разница между ними заключается в том, что первая пробуждает только один, самый приоритетный поток из ждущих выполнения условия, а вторая пробуждает все потоки, ожидающие выполнения условия.

Однако необходимо помнить про специфику ожидания внутри критической секции: вызов функции `pthread_cond_broadcast()` только переведет ожидающие потоки из состояния блокирования на условной переменной в состояние блокировки на мьютексе, поскольку мьютекс сможет захватить только самый приоритетный поток.

Нештатным способом завершения ожидания на условной переменной может быть приход немаскированного сигнала `UNIX`. Если для данного сигнала определен обработчик, он выполнится без захвата мьютекса, а попытка захвата будет произведена уже после его завершения.

Выполнение условия для единичного потока:

```
int pthread_cond_signal(pthread_cond_t* cond);
```

Функция переводит в состояние готовности самый приоритетный поток из заблокированных на условной переменной `cond`, после чего поток предпринимает попытку захвата своего мьютекса. Если есть несколько потоков с равным (и высшим) приоритетом, заблокированных на условной переменной, то разблокируется тот поток, который ожидал дольше остальных.

Выполнение условия для всех ожидающих потоков:

```
int pthread_cond_broadcast(pthread_cond_t* cond);
```

Вызов функции разблокирует все потоки, заблокированные на условной переменной `cond`. Потоки разблокируются в порядке приоритетов. Для потоков равного приоритета разблокирование проводится в порядке `FIFO`.

Разрушение условной переменной:

```
int pthread_cond_destroy(pthread_cond_t* cond);
```

Вызов функции деинициализирует условную переменную `cond`. Для дальнейшего использования условной переменной, на которую ссылается `cond`, ее необходимо инициализировать вызовом `pthread_cond_init()`. Функция может использоваться для изменения параметров условной переменной.

Пример 2.


```

#include<stdio.h>
#include <pthread.h>
int data_ready = 0;
int inf = 0;
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t condvar = PTHREAD_COND_INITIALIZER;
void *consumer (void * notused)
{
    printf("This is the consumer \n");
    while(1)
    {
        pthread_mutex_lock (&mutex);
        printf("W1 \n");
        while (!data_ready)
        {
            printf("W2 \n");
            pthread_cond_wait (&condvar, & mutex);
            printf("W3 \n");
        }
        printf("Data from the producer, times = %d \n",inf);
        data_ready=0;
        pthread_cond_signal(&condvar);
        pthread_mutex_unlock(&mutex);
    }
}

void *producer (void * notused)
{
    printf("This is the producer \n");
    while(1)
    {
        sleep(2);
        printf("The producer has received data from the
consumer, times = %d \n",inf);
        pthread_mutex_lock (&mutex);
        printf("Wp1 \n");
        while (data_ready)
        {
            printf("Wp2 \n");
            pthread_cond_wait (&condvar, & mutex);
        }
        data_ready=1;
        inf++;
        printf("Wp3 \n");
        pthread_cond_signal(&condvar);
        pthread_mutex_unlock(&mutex);
    }
}

main()
{
    printf("Start \n");
    pthread_create(NULL,NULL, consumer,NULL);
    pthread_create(NULL,NULL, producer,NULL);
    sleep(10);
}

```



```
ttyp0: sh
# gcc condvar.c -o condvar
# ./condvar
Start
This is consumer
Wc1
Wc2
This is producer
The producer has received data from the consumer, times = 0
Wp1
Wp3
Wc3
Data from the producer, times = 1
Wc1
Wc2
The producer has received data from the consumer, times = 1
Wp1
Wp3
Wc3
Data from the producer, times = 2
Wc1
Wc2
The producer has received data from the consumer, times = 2
Wp1
Wp3
Wc3
Data from the producer, times = 3
Wc1
Wc2
The producer has received data from the consumer, times = 3
Wp1
Wp3
Wc3
Data from the producer, times = 4
Wc1
Wc2
# _
```

2.4.4 Барьеры

Барьер предназначен для ожидания условия достижения несколькими заданными потоками точки синхронизации. Достигнув этой точки, потоки освобождаются “одновременно” и уже с этой точки продолжают свое независимое развитие.

Заголовочный файл `<pthread.h>` так описывает тип `pthread_barrier_t`:

```
typedef struct
{
    unsigned int barrier;
    unsigned int count;
    pthread_rmutex_t lock;
    pthread_cond_t bcond;
} pthread_barrier_t;
```

Согласно документации [QNX 6.2.1](#) все функции работы с барьером и его атрибутами описаны в заголовочном файле `<pthread.h>`, за исключением двух функций `pthread_barrier_wait()` и `pthread_barrierattr_setpshared()`, о которых говорится, что они описаны в файле `<sync.h>`. Но если заглянуть в заголовочные файлы, то выясняется, что можно спокойно использовать для абсолютно всех функций работы с барьером либо заголовочный файл `<pthread.h>`, либо `<sync.h>`.

Создание барьера:

```
int pthread_barrier_init (pthread_barrier_t *barrier, const
pthread_barrierattr_t *attr, unsigned int count);
```

Создается объект типа барьер по заданному адресу (указатель на барьер находится в аргументе `barrier`) и с атрибутами, указанными в `attr`. Аргумент `count` задает число потоков, которые должны вызвать функцию `pthread_barrier_wait()`.

Инициализация блока параметров барьера:

```
int pthread_barrierattr_init(pthread_barrierattr_t* attr);
```

Функция разрушения барьера:

```
int pthread_barrier_destroy(pthread_barrier_t* barrier);
```

после чего соответствующий элемент синхронизации `barrier` не может использоваться без повторной его инициализации.

Функция разрушения параметров барьера:

```
int pthread_barrierattr_destroy(pthread_barrierattr_t* attr)
```

Параметры барьера описываются типом `pthread_barrierattr_t` и аналогично другим типам синхронизации используются в момент инициализации элемента синхронизации. Фактически единственным атрибутом барьера является модификатор доступа к барьеру из других процессов:

```
int pthread_barrierattr_setpshared(pthread_barrierattr_t*  
attr, int pshared);
```

```
int pthread_barrierattr_getpshared(const  
pthread_barrierattr_t* attr, int* pshared);
```

По умолчанию атрибуты барьера запрещают доступ к элементу синхронизации из других процессов. Для того чтобы изменить возможность доступа к созданному ранее барьеру, его необходимо разрушить, установить соответствующий атрибут и инициализировать барьер повторно.

Ожидание на барьере:

```
int pthread_barrier_wait (pthread_barrier_t *barrier);
```

Когда поток вызывает `pthread_barrier_wait()`, он блокируется до тех пор, пока число потоков, вызвавших функцию `pthread_barrier_wait()` (и тем самым заблокированных) не станет равным числу, которое изначально задано в параметре `count` функции `pthread_barrier_init()`. После того, как заданное число потоков вызовет `pthread_barrier_wait()`, все потоки разблокируются одновременно.

Необходимо с особой осторожностью относиться к использованию барьеров для остановки и синхронизации потоков разных приоритетов. Поскольку потоки, ожидающие у барьера, находятся в блокировке на условной переменной (внутренней), то система никак не отслеживает эффекты, связанные с возможной инверсией приоритетов.

Если поток, заблокированный на барьере, получает сигнал, для которого определен обработчик, то обработчик сигнала выполняется, но по завершении его выполнения поток вновь блокируется до выполнения условия барьера.

Документация [QNX](#) утверждает, что нельзя заранее сказать, какой поток будет освобожден первым, когда заданное количество потоков достигнет барьера. Учитывая, что при реализации операций над потоками использовалась комбинация мьютекса с условной переменной (как видно из определения типа `pthread_barrier_t`), освобождение заблокированных потоков будет проходить по принципам, определенным для потоков, заблокированных на условной переменной и получающих “широковещательное” ([broadcast](#)), или одновременное, освобождение. Документация [QNX](#) утверждает, что в таком случае первым будет “отпущен” тот поток, который ждал дольше других.

Пример 3.

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <time.h>
#include <pthread.h>
#include <sys/neutrino.h>
pthread_barrier_t barrier; // барьер
void *thread1 (void *not_used)
{
    time_t now;
    time (&now);
    printf ("thread1 starting at %s", ctime (&now));
    // какие-то вычисления
    // просто засыпаем
    sleep (20);
    pthread_barrier_wait (&barrier);
    // Здесь все три потока будут завершены
    time (&now);
    printf ("barrier in thread1() done at %s", ctime (&now));
}
void *thread2 (void *not_used)
{
    time_t now;
    time (&now);
    printf ("thread2 starting at %s", ctime (&now));
    // do the computation
    // Просто спим
    sleep (40);
    pthread_barrier_wait (&barrier);
    // Здесь все три потока уже завершат свое исполнение
    time (&now);
    printf ("barrier in thread2() done at %s", ctime (&now));
}

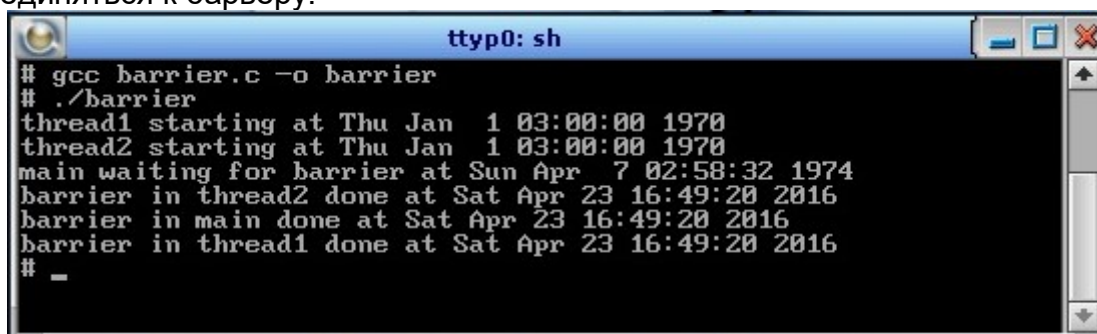
int main () // ignore arguments
{
    time_t now;
    // Создаем барьер на три потока
    pthread_barrier_init (&barrier, NULL, 3);
    // Стартуем два потока
    pthread_create (NULL, NULL, thread1, NULL);
    pthread_create (NULL, NULL, thread2, NULL);
    // в этой точке thread1 и thread2 выполняются
    time (&now);
    printf ("main() waiting for barrier at %s", ctime (&now));
    pthread_barrier_wait (&barrier);
    // после этой точки все три потока завершатся
    time (&now);
    printf ("barrier in main() done at %s", ctime (&now));
    pthread_exit(NULL);
    return (EXIT_SUCCESS);
}

```

Основной поток создает барьер и инициализирует общее число потоков, которые должны быть синхронизированы с помощью барьера, после чего они смогут продолжить работу. В приведенном выше примере, мы использовали три потока: один основной поток `main()`, и два других `thread1()` и `thread2()`.

Затем мы стартуем `thread1()` и `thread2()`. Чтобы упростить пример, была использована временная задержка, для имитации расчетов. Для синхронизации основной поток просто блокируется на этом барьере, зная, что

барьер будет разблокирован только после того, как два работающих потока присоединяться к барьеру.



```
# gcc barrier.c -o barrier
# ./barrier
thread1 starting at Thu Jan 1 03:00:00 1970
thread2 starting at Thu Jan 1 03:00:00 1970
main waiting for barrier at Sun Apr 7 02:58:32 1974
barrier in thread2 done at Sat Apr 23 16:49:20 2016
barrier in main done at Sat Apr 23 16:49:20 2016
barrier in thread1 done at Sat Apr 23 16:49:20 2016
# -
```

2.4.5 Сигналы (Signals)

Сигналы инициируются некоторыми событиями в системе и посылаются процессу для его уведомления о том, что произошло нечто неординарное, требующее определенной реакции. Порождающее сигнал событие может быть действием пользователя или может быть вызвано другим процессом или ядром операционной системы. Сигналы являются одним из самых старых и традиционных механизмов [UNIX](#).

Из этого можно заключить, что:

- действия, вызываемые для обработки сигнала, являются принципиально **асинхронными**;
- сигналы могут быть использованы как простейшее, но мощное средство межпроцессного взаимодействия.

Все сигналы определяются целочисленными константами, но для программиста это в принципе не так важно, поскольку каждому сигналу приписано символическое наименование вида **SIG***. Все относящиеся к сигналам определения находятся в заголовочном файле **<signal.h>**, который должен быть включен в любой код, работающий с сигналами.

Посланный сигнал обрабатывается получившим его потоком или процессом (как уже обсуждалось ранее, собственно процесс ничего не может обрабатывать: это пассивная субстанция; понятно, что речь в таком контексте идет об обработке сигнала главным потоком процесса). Поток может отреагировать на полученный сигнал следующими способами (иногда действие, установленное для конкретного сигнала, называют **диспозицией** сигнала):

- **Стандартное действие** - выполнение действия, предписанного для обработки этого сигнала по умолчанию. Для многих сигналов действием по умолчанию является завершение, но это необязательно; есть сигналы, которые по умолчанию игнорируются. Для большей части сигналов, чьим действием по умолчанию является принудительное завершение процесса, предписано действие создания дампа памяти при завершении (**core dump**), но для некоторых, например **SYSINT** (завершение по **[Ctrl+C]**), определено такой дамп при завершении не создавать.
- **Игнорирование сигнала** - сигнал не оказывает никакого воздействия на ход выполнения потока получателя.
- **Вызов обработчика** - по поступлению сигнала вызывается функция реакции, определенная пользователем. Если для сигнала устанавливается функция-обработчик, то говорят, что сигнал перехватывается (относительно стандартного действия).

Для различных сигналов программа может установить различные механизмы обработки. Более того, в ходе выполнения программа может динамически переопределять реакции, установленные для того или иного сигнала.

Для большинства сигналов их воздействие можно перехватить из

программного кода или игнорировать, но не для всех. Например, сигналы **SIGKILL** и **SIGSTOP** не могут быть ни перехвачены, ни проигнорированы. Другой пример - описываемые далее специальные сигналы **QNX**, начиная с **SIGSELECT** и далее.

ОС **QNX** определяет 64 вида сигналов. Их диапазоны выглядят следующим образом.

Диапазон сигнала	Описание
1 .. 57	57 стандартных сигналов POSIX
41 .. 56	16 POSIX сигналов реального времени (от SIGRTMIN до SIGRTMAX)
57 .. 64	8 специализированных сигналов ОС QNX Neutrino

Восемь специальных сигналов **QNX Neutrino** не могут быть проигнорированы или перехвачены. Попытка вызвать функции **signal()** или **sigaction()** или системный вызов (*kernel call*) **SignalAction()** для изменения этого сигнала будет завершаться с ошибкой **EINVAL**.

Кроме того, эти сигналы всегда маскированы (**blocked**) и для них включенна организация очередей. Попытка демаскировки (**unblock**) этих сигналов посредством функции **sigprocmask()** или системного вызова **SignalProcmask()** будет проигнорирована.

Из восьми специальных сигналов, первые два имеют специальные имена:

```
#define SIGSELECT (SIGRTMAX + 1)
```

```
#define SIGPHOTON (SIGRTMAX + 2)
```

В силу своей недоступности эта группа сигналов представляет небольшой интерес для разработчика приложений.

POSIX-сигналы общего назначения (1...40), из которых рассмотрим только самые употребляемые:

Идентификатор сигнала	Описание
SIGABRT [6]	Сигнал аварийного завершения (<i>process abort signal</i>). Посылается процессу, например, при вызове им функции abort()
SIGALRM [14]	Сигнал наступление тайм-аута таймера сигналов (<i>real-time alarm clock</i>). Посылается при срабатывании ранее установленного пользовательского таймера (таймер устанавливается заданием первого параметра setitimer() , равным ITIMER_REAL), например, при помощи системного вызова alarm()
SIGBUS [10]	Сигнал ошибки на шине (<i>bus error</i>). Этот сигнал посылается при возникновении некоторых аппаратных ошибок (зависит от платформы); обычно он генерируется при попытке обращения к допустимому виртуальному адресу, для которого нет физической страницы. Если ошибка возникла повторно во время того, как процесс уже находится в обработчике сигнала (<i>signal handler</i>) из-за аналогичной ошибки, то процесс будет остановлен
SIGCHLD (или SIGCLD) [18]	Сигнал завершения или остановка дочернего процесса (<i>child process terminated or stopped</i>). По умолчанию родительский процесс игнорирует этот сигнал, поэтому если нужно получать уведомление о завершении порожденного процесса, этот сигнал нужно перехватывать.
SIGCONT [25]	Сигнал продолжения работы остановленного процесса (<i>continue executing if stopped</i>). Это сигнал управления

	выполнением процесса, который возобновляет его выполнение, если ранее он был остановлен сигналом SIGSTOP ; если процесс не остановлен, он игнорирует этот сигнал.
SIGDEADLK [7]	Сигнал о взаимной блокировке на мьютексе (<i>mutex deadlock occurred</i>). Если не было вызова функции SyncMutexEvent() и если возникли условия, при которых ядро должно передать данное событие, тогда ядро передает сигнал SIGDEADLK
SIGEMT [7]	Сигнал EMT -инструкции (<i>emulator trap</i>). Как видно из сравнения с предыдущим сигналом, у них один код, то есть в QNX SIGDEADLK и SIGEMT - это один сигнал.
SIGFPE [8]	Сигнал некорректной арифметической операции (<i>floating point exception</i>), такой как деление на ноль, или результирующее переполнение. Если происходит повторная ошибка, пока процесс находится в обработке сигнала с аналогичной ошибкой, процесс будет остановлен
SIGHUP [1]	Сигнал освобождения линии, разрыв связи с управляющим терминалом (<i>hangup signal</i>). Посылается всем процессам, подключенным к управляющему терминалу при отключении терминала (обычно управляющий терминал группы процесса является терминалом пользователя, но это не всегда так). Этот сигнал также посылается всем членам сеанса, если завершает работу лидер сеанса (обычно процесс командного интерпретатора), связанного с управляющим терминалом (это гарантирует, что, если не были приняты специальные меры, при выходе пользователя из системы будут завершены все запущенные им фоновые процессы)
SIGILL [4]	Сигнал попытки выполнения недопустимой инструкции процессора (<i>illegal instruction</i>). Если ошибка возникает повторно во время того, как процесс находится в обработке прерываний аналогичной ошибки, процесс будет остановлен. Одной из причин возникновения этого сигнала является выполнение операций, которые требуют I/O привилегий. Поток может запросить эти привилегии, вызвав функцию ThreadCtl() , указав флаг _NTO_TCTL_IO : ThreadCtl(_NTO_TCTL_IO, 0);
SIGINT [2]	Интерактивный предупредительный сигнал (<i>break</i>), который генерируется при нажатии [Ctrl+C]
SIGIOT	IOT инструкция (не генерируется на x86 платформе)
SIGKILL [9]	Сигнал уничтожение процесса (<i>kill</i>) должен использоваться только для экстренных ситуаций. Это сигнал безусловного завершения, посылаемый процессу другим процессом или системой (при завершении работы системы). Этот сигнал не может быть перехвачен или проигнорирован
SIGPIPE [13]	Сигнал попытки записать в программный канал без зарегистрированных читателей (принимающий процесс уже завершил работу)
SIGPOLL (или SIGIO) [22]	Сигнал уведомления об одном из программно опрашиваемых событий (<i>pollable event</i>). Этот сигнал генерируется системой, когда некоторый открытый дескриптор файла готов для ввода или вывода.
SIGPROF [29]	Сигнал профилирующего таймера (<i>profiling timer expired</i>). Как и сигнал SIGALRM , этот сигнал возбуждается по истечении

	времени таймера (но это другой таймер), который используется для измерения времени выполнения процесса в пользовательском и системном режимах (таймер устанавливается заданием первого параметра <code>setitimer()</code> , равным <code>ITIMER_PROF</code>).
SIGPWR	Сигнал сбоя питания или перезагрузки
SIGQUIT [3]	Интерактивный сигнал выхода из процесса (<i>quit</i>)
SIGSEGV [11]	Сигнал обращения к некорректному адресу памяти, ошибки защиты памяти, нарушение границ сегмента памяти (<i>invalid memory reference, segmentation violation</i>). Если ошибка возникает повторно во время того, как процесс находится в обработчике прерываний аналогичной ошибки, процесс будет остановлен.
SIGSTOP [23]	Сигнал временной остановки процесса (<i>sendable stop signal not from tty</i>). Если пользователь вводит с терминала <code>[Ctrl+Z]</code> , то активному процессу посылается этот сигнал и процесс приостанавливается. Позже процесс может быть возобновлен с точки остановки при получении сигнала SIGCONT . Этот сигнал не может быть перехвачен или проигнорирован
SIGSYS [12]	Сигнал некорректного аргумента для системного вызова (<i>invalid system call, bad argument to system call</i>)
SIGTERM [15]	Программный сигнал завершения (<i>software termination signal from kill</i>). Программист может использовать этот сигнал для того, чтобы дать процессу время для «наведения порядка», прежде чем посылать ему сигнал SIGKILL . Именно этот сигнал посылается по умолчанию командой <code>kill</code> без параметра, указывающего сигнал
SIGTRAP [5]	Сигнал трассировочного прерывания (<i>trace trap</i>). Это особый сигнал, который в сочетании с системным вызовом <code>ptrace()</code> используется отладчиками: <code>sdb</code> , <code>adb</code> , <code>gdb</code> . По умолчанию сигнал приводит к аварийному завершению. Обработка сигнала не сбрасывается, когда он перехватывается
SIGTSTP [24]	Терминальный сигнал остановки (<i>terminal stop signal</i>). Генерируется при нажатии специальной комбинации остановки <code>[Ctrl+Z]</code> . Аналогичен сигналу SIGSTOP , но, в отличие от последнего, может быть перехвачен или проигнорирован
SIGTTIN [26]	Сигнал остановки фонового процесса, если он пытается прочитать данные со своего управляющего терминала (<i>background process attempting read</i>)
SIGTTOU [27]	Сигнал остановки фонового процесса, если он пытается писать данные на свой управляющий терминал (<i>background process attempting write</i>)
SIGURG [21]	Сигнал о поступлении в буфер сокета срочных (приоритетных) данных (<i>high bandwidth data is available at a socket, urgent condition on I/O channel</i>). Уведомляет процесс, что по открытому им сетевому соединению получены внеочередные данные
SIGUSR1 [16]	Зарезервированный сигнал пользователя
SIGUSR2 [17]	Зарезервированный сигнал пользователя
SIGVTALRM [28]	Сигнал виртуального таймера (<i>virtual timer expired</i>). Подобно SIGPROF и SIGALRM , этот сигнал возбуждается по истечении времени таймера (это третий из доступных таймеров), который измеряет время процессора только в пользовательском

	режиме(таймер устанавливается заданием первого параметра <code>setitimer()</code> , равным <code>ITIMER_VIRTUAL</code>).
SIGWINCH [20]	Сигнал, который генерируется (в консольном режиме <code>pterm</code> и <code>xterm</code> эмулируют его вручную при изменении их размеров) при изменении размера окна (<i>window size change</i>) для запущенного в окне приложения, чтобы оно перерисовало свой экран вывода
SIGXCPU [30]	Сигнал о превышении лимита процессорного времени (<i>CPU time limit exceeded</i>). Посылается процессу при исчерпании им ранее установленного лимита процессорного времени. Действие по умолчанию - аварийное завершение.
SIGXFSZ [31]	Сигнал о превышении предела, установленного на размер файла (<i>file size limit exceeded</i>). Действие по умолчанию - аварийное завершение.

Модель надежных сигналов

В новой модели обработки сигналов (называемой еще моделью надежных сигналов) используются не единичные сигналы, а наборы сигналов - тип `sigset_t`.

Для формирования сигнальных наборов определяется набор специальных операций:

`sigemptyset(sigset_t *set)`

инициализирует набор `set`, исключая из него все сигналы;

`sigfillset(sigset_t *set)`

инициализирует набор `set`, включая в него все сигналы;

`sigaddset(sigset_t *set, int signo)`

добавляет в инициализированный набор `set` единичный сигнал `signo`;

`sigdelset(sigset_t *set: int signo)`

удаляет из инициализированного набора `set` единичный сигнал `signo`.

В качестве `signo` в функциях добавления и удаления единичных сигналов используется символическая константа, соответствующая сигналу (такая как `SIGINT`), либо численное значение сигнала, но в этом случае код становится зависимым от системы. Легко увидеть, что, пользуясь совокупностью этих 4-х операций, можно сформировать любой произвольный набор сигналов. Например:

```
sigset_t sig;
sigemptyset(&sig);
sigaddset(&sig, SIGPOLL);
sigaddset(&sig, SIGALRM);
```

Этот фрагмент кода формирует сигнальный набор, состоящий из двух сигналов: `SIGPOLL` и `SIGALRM`.

Диспозиция обработки каждого сигнала в этой модели устанавливается функцией:

```
int sigaction(int signo, const struct sigaction *act, struct sigaction *oact);
```

где `signo` - номер (имя) сигнала, для которого устанавливается диспозиция;

`act` - определение нового обработчика сигнала;

`oact` - структура (если указано не `NULL`), где будет сохранено описание ранее установленного обработчика (например, для последующего восстановления реакции).

Структура описания обработчика `sigaction` определена так:

```
struct sigaction
{
```

```

#define sa_handler un _sa_handler
#define sa_sigaction un _sa_sigaction union {
void (*_sa_handler) (_SIG_ARGS);
void (*_sa_sigaction)(int, siginfo_t*, void*),
} un;
Int sa_flags;
sigset_t sa_mask;
};

```

Первое поле **sa_handler** определяет обработчик, устанавливаемый для сигнала в традиционной модели. Это может быть:

- **SIG_DFL** - восстановить обработчик сигнала, принятый по умолчанию (определения **SIG_DFL** и **SIG_IGN**);
- **SIG_IGN** - игнорировать данный сигнал;
- адрес функции-обработчика, устанавливаемой как реакция на поступление этого сигнала. Эта функция будет выполняться при поступлении сигнала **signo**, и в качестве аргумента вызова она получит значение **signo** (одна функция может выступать как обработчик целой группы сигналов). Управление будет передано этой функции, как только процесс получит сигнал, какой бы участок кода при этом ни выполнялся. После возврата из функции управление будет возвращено в ту точку, в которой выполнение процесса было прервано.

Второе поле **sa_mask** демонстрирует первое применение набора сигналов: сигналы, установленные в **sa_mask**, будут блокироваться на время выполнения обработчика **sa_handler** (при вызове **sa_handler** и сам сигнал **signo** будет неявно добавлен в набор **sa_mask**, поэтому его можно не указывать явно). Это не значит, что поступившие в это время сигналы будут игнорироваться и теряться, просто их обработка будет отложена до завершения работы обработчика **sa_handler**.

Поле **sa_flags** может использоваться для изменения характера реакции на сигнал **signo**. Возможны следующие значения поля флагов:

- **SA_RESETHAND** - после выполнения функции обработчика будет восстановлен обработчик по умолчанию (**SIG_DFL**, что соответствует духу модели “ненадежных сигналов” и позволяет воспроизводить ее поведение);
- **SA_NOCLDSTOP** - используется только для сигнала **SIGCHLD**; флаг указывает системе не генерировать для родительского процесса **SIGCHLD** от порожденных процессов, которые завершаются посредством **SIGSTOP**.
- **SA_SIGINFO** - при этом будет использована обработка сигналов на базе очереди сигналов (модель сигналов реального времени). По умолчанию используется простая обработка: результат воздействия нескольких сигналов определяется последним поступившим. В случае установки этого флага будет использована расширенная форма обработчика **sa_sigaction** (при этом поле **sa_handler** не будет использоваться). Обработчику будет передаваться дополнительная информация о сигнале - структура **siginfo_t** (его номер, **PID** пославшего сигнал процесса, действующий идентификатор пользователя этого процесса). Эта весьма объемная структура будет очень кратко рассмотрена ниже. Ее описание вынесено в отдельный заголовочный файл **<sys/siginfo.h>** и может быть изучено там.

Еще один вызов **API**-сигналов, который широко используется в этой и последующих моделях обработки (сигналы реального времени, реакция в потоках):

```

int sigprocmask(int how, const sigset_t *set, sigset_t
*oset);

```

Этот вызов позволяет прочитать текущее значение (если **set** установлено в **NULL**, то параметр **how** игнорируется) или переустановить сигнальную маску для

текущего потока. Параметры вызова:

- **set** - это то значение, в соответствии с которым корректируется сигнальная маска процесса;

- **how** - указывает, какое именно действие переустановки сигнальной маски требуется осуществить:

- **SIG_BLOCK** - добавить сигналы, указанные в **set** к маске процесса (заблокировать реакцию на эти сигналы);

- **SIG_UNBLOCK** - сбросить указанные **set** сигналы в сигнальной маске;

- **SIG_SETMASK** - переустановить сигнальную маску процесса на значение, указанное в **set**.

- **oset** - значение, в котором будет сохранено значение маски, предшествующее вызову (старое значение).

Как и большинство сигнальных функций, данная функция возвращает нулевое значение в результате успешного выполнения и **-1** в случае неудачи, при этом код ошибки устанавливается в **errno**.

Именно эта функция снимает одно из самых существенных ограничений, свойственных модели “ненадежных сигналов”, - позволяет заблокировать реакцию на сигналы при выполнении критических участков кода и восстановить ее при завершении выполнения этих участков.

3 Содержание отчета по лабораторной работе

Отчет по лабораторной работе должен содержать следующие сведения:

цель работы;

примеры выполнения команд;

пояснительные комментарии к командам.